

Editing Files in Kali Linux with `nano`

A practical guide for making changes to Python scripts (like `uto_ligo_analysis.py`) from the terminal — written so you can describe exactly what you want changed, and do it yourself with confidence.

1. Opening a file in nano

`nano` is a simple text editor that runs right in the terminal — no separate window, no mouse required.

```
nano ~/Projects/Ligo_Analysis/uto_ligo_analysis.py
```

This opens the file. You'll see the file's contents fill the terminal, with a menu of shortcuts at the bottom (things like `^O Write Out`, `^X Exit` — the `^` symbol means **Ctrl**).

Key things to know before you touch anything:

- You can move around with the **arrow keys** — no mouse needed.
- Whatever you type is inserted right where your cursor is, like any normal text editor.
- Nothing is saved until you explicitly tell it to (Section 4).

2. Finding the part you want to change

Scrolling manually through a long script is slow and error-prone. Use nano's built-in search instead.

To search:

1. Press `Ctrl + W` (this is "Where is")
2. Type the text you're looking for — e.g. `local_csv_dir` or `N_CONTROL_SHUFFLES`
3. Press `Enter`

Nano jumps to the first match. If it's not the one you want, press `Ctrl + W` then `Enter` again to jump to the next match.

Tip: find the line number *before* opening the file

If you want to know exactly where something is before you even open nano, use `grep` from the terminal:

```
grep -n "N_CONTROL_SHUFFLES" ~/Projects/Ligo_Analysis/uto_ligo_analysis.py
```

- `grep` searches text inside a file for a pattern.
- `-n` tells it to show the **line number** next to each match.
- This is useful because it tells you *before* you open the editor exactly where to look, and it's also the fastest way to tell me "the code I want changed is on line 812."

Example output:

```
812:N_CONTROL_SHUFFLES = 1000
1187:     for shuffle_i in range(N_CONTROL_SHUFFLES):
```

Once you're in nano, you can jump straight to a line number with:

`Ctrl + _` (hold Ctrl and press the underscore/minus key)

Then type the line number and press `Enter`.

3. Making the edit

Once your cursor is where you need it:

- **To type new text:** just start typing — it inserts at the cursor position.
- **To delete a character:** `Backspace` (deletes before cursor) or `Delete` (deletes after cursor), same as normal.
- **To delete a whole line:** put your cursor anywhere on the line and press `Ctrl + K` (this "cuts" the line — careful, it's gone from view immediately, though see the undo tip below).
- **To paste a line you just cut:** `Ctrl + U` (pastes it back wherever your cursor is).
- **To undo a mistake:** `Alt + U` (nano does support undo).
- **To redo:** `Alt + E`.

A note on indentation (important for Python!):

Python uses indentation (spaces at the start of a line) to know what belongs inside a function, loop, or `if` statement. When editing a line, keep the same number of leading spaces as the line had before, unless you specifically mean to change the structure. If you're replacing a whole line, it's often safest to:

1. Go to the start of the line (`Ctrl + A` moves to line start in nano — not "select all" like in most programs)
2. Select to the end of the line (`Ctrl + K` cuts the entire line if your cursor is at the start)
3. Type the replacement, matching the original indentation

4. Saving and exiting

- **Save (without exiting):** `Ctrl + O` (that's the letter O, for "Write Out"), then press `Enter` to confirm the filename it suggests.
- **Exit nano:** `Ctrl + X`
- **Save and exit in one go:** `Ctrl + O`, `Enter`, then `Ctrl + X`

If you try to exit with unsaved changes, nano will ask `Save modified buffer?` — press `Y` for yes, `N` to discard your changes, or `Ctrl + C` to cancel and go back to editing.

5. Double-checking your change worked

After saving, you can print just that section of the file back to the terminal to confirm it looks right, without reopening the whole editor:

```
sed -n '810,815p' ~/Projects/Ligo_Analysis/uto_ligo_analysis.py
```

- `sed -n '810,815p'` prints only lines 810 through 815.
- Swap in whatever line range covers your edit.

This is a good habit before running a long script — a 30-second check here can save you from an hour-long run failing on a typo.

6. Running the script after editing

```
python3 ~/Projects/Ligo_Analysis/uto_ligo_analysis.py
```

If it errors immediately, the error message will usually include a line number — that tells you exactly where to go back and look with `grep -n` or `sed -n`.

7. Quick reference — the shortcuts that matter most

Action	Shortcut
Search for text	<code>Ctrl + W</code>
Jump to a specific line number	<code>Ctrl + _</code> then type the number
Move to start of current line	<code>Ctrl + A</code>
Move to end of current line	<code>Ctrl + E</code>

Action	Shortcut
Cut current line	Ctrl + K
Paste last cut line	Ctrl + U
Undo	Alt + U
Redo	Alt + E
Save	Ctrl + O , then Enter
Exit	Ctrl + X

8. How to describe a change to me efficiently

Now that you know the tools, when you want help changing something in a script, the fastest way to tell me what you want is:

1. Run `grep -n "keyword" filename.py` to find the relevant line number(s).
2. Tell me the line number(s) and what you want changed — e.g. *"line 812 sets `N_CONTROL_SHUFFLES` to 1000, I want to drop it to 100 for a faster test run."*
3. I can give you the exact replacement line, and you paste it in using the search-and-edit steps above.

This skips a lot of back-and-forth screenshotting and gets us straight to the fix.